

SKILL WEEK 7

Task 1: Input Redirection

To Parse Input Redirection, Open Files, Redirect stdin, Handle Missing Files, Restore Streams, Test File Inputs

To read input from a file by redirecting standard input using `dup2()` and handle missing input files.

Program

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

int main() {
    int fd;
    pid_t pid;

    fd = open("input.txt", O_RDONLY);

    if (fd < 0) {
        perror("input.txt");
        return 1;
    }

    pid = fork();

    if (pid < 0) {
        perror("fork");
        close(fd);
        return 1;
    }

    if (pid == 0) {
        dup2(fd, STDIN_FILENO);
        close(fd);

        execlp("cat", "cat", NULL);

        perror("exec");
        exit(1);
    }
}
```

```
close(fd);
waitpid(pid, NULL, 0);

printf("Input redirection completed successfully.\n");

return 0;
}
```

Input

The input is stored in:

input.txt

Contents:

Hello from input file

Output

```
amrutha@blue:~$ mkdir -p ~/SKILL_Week7
amrutha@blue:~$ cd ~/SKILL_Week7
amrutha@blue:~/SKILL_Week7$ echo "Hello from input file" > input.txt
amrutha@blue:~/SKILL_Week7$ cat input.txt
Hello from input file
amrutha@blue:~/SKILL_Week7$ nano input_redirect.c
amrutha@blue:~/SKILL_Week7$ gcc input_redirect.c -o input_redirect
amrutha@blue:~/SKILL_Week7$ ./input_redirect
Hello from input file
Input redirection completed successfully.
```

Test Missing File

```
amrutha@blue:~/SKILL_Week7$ rm input.txt
amrutha@blue:~/SKILL_Week7$ ./input_redirect
input.txt: No such file or directory
```

Output:

input.txt: No such file or directory

Observation

The file was opened using `open()` and redirected to standard input using `dup2()`. The child process successfully read the file contents through `stdin`. A missing file was handled using an error message.

Result

Input redirection using `stdin` was successfully implemented and tested.

Task 2: Output Redirection

To Parse Output Redirection, Create Output Files, Redirect stdout, Handle Permissions, Verify Output Data, Test Redirection

Program

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

int main() {
    int fd;
    pid_t pid;

    fd = open("output.txt",
              O_WRONLY | O_CREAT | O_TRUNC,
              0644);

    if (fd < 0) {
        perror("output.txt");
        return 1;
    }

    pid = fork();

    if (pid < 0) {
        perror("fork");
        close(fd);
        return 1;
    }

    if (pid == 0) {
        dup2(fd, STDOUT_FILENO);
        close(fd);

        execlp("echo", "echo",
               "Hello from output redirection",
               NULL);

        perror("exec");
        exit(1);
    }

    close(fd);
    waitpid(pid, NULL, 0);

    printf("Output redirection completed successfully.\n");

    return 0;
}
```

Input

No keyboard input is required.

The program internally sends:

Hello from output redirection

to stdout, which is redirected to output.txt.

Output

```
amrutha@blue:~/SKILL_Week7$ nano output_redirect.c
amrutha@blue:~/SKILL_Week7$ gcc output_redirect.c -o output_redirect
amrutha@blue:~/SKILL_Week7$ ./output_redirect
Output redirection completed successfully.
amrutha@blue:~/SKILL_Week7$ cat output.txt
Hello from output redirection
```

Observation

Standard output was redirected from the terminal to output.txt using dup2(). The file was created using O_CREAT, and existing contents were replaced using O_TRUNC.

Result

Output redirection was successfully implemented and the redirected data was verified.

Task 3: Append Redirection

To Parse Append Operator, Open Files in Append Mode, Redirect Streams, Preserve Existing Data, Validate Appends, Test Scenarios

To redirect output to a file in append mode while preserving the existing contents.

Program

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

int main() {
    int fd;
    pid_t pid;

    fd = open("append.txt",
              O_WRONLY | O_CREAT | O_APPEND,
              0644);

    if (fd < 0) {
        perror("append.txt");
        return 1;
    }
}
```

```

    pid = fork();

    if (pid < 0) {
        perror("fork");
        close(fd);
        return 1;
    }

    if (pid == 0) {
        dup2(fd, STDOUT_FILENO);
        close(fd);

        execlp("echo", "echo",
               "New line appended to file",
               NULL);

        perror("exec");
        exit(1);
    }

    close(fd);
    waitpid(pid, NULL, 0);

    printf("Append operation completed successfully.\n");

    return 0;
}

```

Terminal Commands

```

amrutha@blue:~/SKILL_Week7$ echo "Existing data" > append.txt
amrutha@blue:~/SKILL_Week7$ cat append.txt
Existing data
amrutha@blue:~/SKILL_Week7$ nano append_redirect.c
amrutha@blue:~/SKILL_Week7$ gcc append_redirect.c -o append_redirect
amrutha@blue:~/SKILL_Week7$ ./append_redirect
Append operation completed successfully.
amrutha@blue:~/SKILL_Week7$ cat append.txt
Existing data
New line appended to file
amrutha@blue:~/SKILL_Week7$ ./append_redirect
Append operation completed successfully.
amrutha@blue:~/SKILL_Week7$ cat append.txt
Existing data
New line appended to file
New line appended to file

```

Output

Before running:

Existing data

After running:

Existing data

New line appended to file

Test Again

Run:

./append_redirect

cat append.txt

Output:

Existing data

New line appended to file

New line appended to file

Observation

The file was opened using O_APPEND, so new data was added to the end of the file without removing the existing contents.

Result

Append redirection was successfully implemented, and existing file data was preserved.

Task 4: Error Redirection

To Parse stderr Redirection, Redirect Error Streams, Capture Error Output, Verify File Contents, Handle Failures, Test Error Cases

To redirect standard error to a file and capture error messages generated by a failed command.

Program

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

int main() {
    int fd;
    pid_t pid;
```

```

fd = open("error.txt",
          O_WRONLY | O_CREAT | O_TRUNC,
          0644);

if (fd < 0) {
    perror("error.txt");
    return 1;
}

pid = fork();

if (pid < 0) {
    perror("fork");
    close(fd);
    return 1;
}

if (pid == 0) {
    dup2(fd, STDERR_FILENO);
    close(fd);

    execlp("ls", "ls", "missing_file.txt", NULL);

    perror("exec");
    exit(1);
}

close(fd);
waitpid(pid, NULL, 0);

printf("Error redirection completed successfully.\n");

return 0;
}

```

Input

No keyboard input is required.

The program attempts to execute:

ls missing_file.txt

Since the file doesn't exist, ls generates an error.

Output

```

amrutha@blue:~/SKILL_Week7$ cd ~/SKILL_Week7
amrutha@blue:~/SKILL_Week7$ nano error_redirect.c
amrutha@blue:~/SKILL_Week7$ gcc error_redirect.c -o error_redirect
amrutha@blue:~/SKILL_Week7$ ./error_redirect
Error redirection completed successfully.
amrutha@blue:~/SKILL_Week7$ cat error.txt
ls: cannot access 'missing_file.txt': No such file or directory

```

Observation

The standard error stream was redirected from the terminal to error.txt using `dup2()` with `STDERR_FILENO`. The error generated by the invalid `ls` command was successfully captured in the file.

Result

Standard error redirection was successfully implemented and the generated error message was stored and verified.